

TypeScript練習



if判斷式

當需要比較兩個或者多個值時我們就會使用到判斷式(if)，裡面可以用來比較是否相等或者數字誰大誰小，如下圖左判斷式中的條件為如果 $12 > 10$ 成立的話就會將12打印出來，反之如果不成立就會進到另一個else中，如果你有多個條件可以在if的條件中用&&加入第二個條件(如右圖)，注意右圖需要兩個條件"都"成立這個判斷式才會成立。

```
if (12 > 10) {  
  console.log(12);  
} else {  
  console.log(12);  
}
```

```
if ((12 > 10) && (11 > 10)) {  
  console.log(12);  
} else {  
  console.log(12);  
}
```

類型轉換(文字轉數字)

相信大家寫code的時候很常遇到一個問題是我要計算但是有個變數是string格式沒辦法寫程式計算(系統會顯示錯誤)，這時候我們就需要使用到類型轉換(如下圖)。

```
// 網頁一開始就會執行的生命週期的方法
ngOnInit(): void {
  // console.log開發者用來除錯或是觀察程式有沒有執行到的一個工具
  let numData: number = 3;
  let stringData: string = '2';
  // parseInt這個系統預設的方法 會將()內 你提供的string資料轉換為數字丟出來
  console.log(numData * parseInt(stringData));
}
```

類型轉換(數字轉文字)

那如果我們要將數字轉成文字呢？有兩種辦法。一種也是使用類型轉換的方法String()，另一種就是宣告一個string的變數別且在數字變數前或後加上其他文字或者"，兩種方法都是可執行的。

```
ngOnInit(): void {  
  let num = 1;  
  let num2 = '2';  
  let newString = num2 + '';  
  console.log(newString);  
  console.log(String(num));  
}
```

JSON

JSON (JavaScript Object Notation) 是一種文字式格式，能夠以人類可讀且機器可剖析的方式儲存和交換資料。我們開發前端時時常會接收到後端來的資料中有著JSON的資料，而JSON的資料類型可以為物件、陣列、字串、數字跟空值。

```
{"name": "Allen", "lev": 2, "props": [1, 2, 3], "def": null}
```

類型轉換(物件轉JSON)

我們在寫程式的過程中常常會遇到物件格式的資料但是如果要回傳給後台儲存通常會使用JSON格式，所以我們要回傳資料時就需要先將物件檔案轉為JSON，要如何知道轉換有沒有成功可以看log出來的資料，上面可以展開的就是物件格式下面是JSON格式。

```
let jsonData = {  
  name: '213',  
  lev: 12  
};  
console.log(jsonData);  
console.log(JSON.stringify(jsonData));
```

```
▶ {name: '213', lev: 12}  
{"name":"213","lev":12}
```

類型轉換(JSON轉物件)

反之我們收到後台資料後需要將JSON轉回物件才能使用，不過通常用HttpClient呼叫Api後會自動將收到的資料JSON轉換為物件。

```
ngOnInit(): void {  
  let stringData = '{"name":"213","lev":12}';  
  console.log(stringData);  
  console.log(JSON.parse(stringData));  
}
```

```
{"name":"213","lev":12}
```

```
▶ {name: '213', lev: 12}
```

運算符

我們在寫程式的時候時常會使用到運算，除了基礎的加+、減-、乘*、除/、餘數% 還有其他的判斷運算符下圖是展示基礎的加減乘除還有餘數。

運算	運算子	例子
加法	+	i + 9
減法	-	a - c
乘法	*	a * b
除法	/	x / y
餘數	%	r % s
遞增	++	++s
遞減	--	--s
負數	-	-s

```
let num = 2 + 3; // 5
let num2 = 3 - 2; // 1
let num3 = 2 * 3; // 6
let num4 = 6 / 2; // 3
let num5 = 6 % 4; // 2
```


指派運算符

接下來我們看到指派運算符，這邊的寫法是一種簡寫 (例如 `i += 9` 就是 `i = i + 9` 的意思)。

運算	運算子	例子
指派	=	<code>i = 50</code>
加法指派	+=	<code>i += 9</code>
減法指派	-=	<code>i -= 9</code>
乘法指派	*=	<code>i *= 9</code>
除法指派	/=	<code>i /= 9</code>
餘數指派	%=	<code>i %= 9</code>

```
let num = 2;  
num = 50; // 50  
num += 2 // 52  
num -= 2 // 50  
num *= 2 // 100  
num /= 2 // 50  
num %= 6 // 2
```

比較運算符

接下來我們看到比較運算符，這個通常是用來比較兩個值，通常是使用在if判斷中。

運算	運算子	例子
相等	==	i == 9
不等於	!=	a != c
大於	>	a > b
小於	<	x < y
大於或等於	>=	i >= 0
小於或等於	<=	i <= 9

邏輯運算符

接下來我們看到邏輯運算符，這個通常是使用在if判斷中有多種判斷條件。

運算	運算子	例子
AND	&&	(x>0)&&(y>0)
OR		(a>0) (b>0)
NOT	!	!(a>=b)

```
// 判斷num是否大於10 跟 小於15 不成立
```

```
if (num > 10 && num < 15)
```

```
// 判斷num是否大於10 或者 小於5 成立
```

```
if (num > 10 || num < 5)
```

```
// 判斷num大於10 跟 小於15的相反 因為前面有個！ 成立
```

```
if (!(num > 10 && num < 15))
```

字串常用函數

以下會教各位字串常使用到的函數。

length 這個是可以顯示字串的長度，

slice(起始, 終止) 這個是可以讓你抓取字串內容依照你的設定的起始數字跟終止的數字內的

indexOf() 裡面輸入你想要收尋的內容，會顯示有對應到的位置

```
let string = 'test';  
console.log(string.length); //4  
console.log(string.slice(1,2)); //e  
console.log(string.indexOf('e')); //1
```

數組常用函數

以下會教各位數組(Array)常使用到的函數。

length 這個是可以顯示數組內有多少筆資料

splice(起始, 長度) 從起始開始刪除指定長度的資料

filter 篩選 可以根據你設定的條件回傳內容，條件需要為一個布林值(true/false)

```
let arrayTest = ['a', 'b', 'c', 'd'];
console.log(arrayTest.length); // 4
arrayTest.splice(1, 1);
console.log(arrayTest); // ['a', 'c', 'd']
arrayTest = arrayTest.filter((res) => {
  return res == 'a' || res == 'c';
})
console.log(arrayTest); // ['a', 'c']
```

數組常用函數

push增加一筆資料進陣列中。

pop刪除陣列中最後一筆資料。



```
let arrayData = [];  
arrayData.push('123');  
console.log(arrayData); // ['123']  
arrayData.pop();  
console.log(arrayData); // []
```

數組常用函數

- forEach 循環，跟for迴圈有點類似差異在forEach無法中斷並且無法修改內容(如左圖)
- map 跟forEach很類似，差異在map會創建一個新的資料(如右圖)

```
let array = ['a', 'b', 'c',]
let array2 = array.forEach((item, index, array) => {
  array[index] = item + '+d'
})
console.log(array2) // undefined 不會額外回傳值
```

```
let array = ['a', 'b', 'c',]
array.forEach((item, index, array) => {
  array[index] = item + '+d'
})
console.log(array) // [ 'a+d', 'b+d', 'c+d' ] 原陣列改變了
```

```
let array = ['a', 'b', 'c',]

let array2 = array.map(item => item + '+d')

console.log(array) // [ 'a', 'b', 'c' ]
console.log(array2) // [ 'a+d', 'b+d', 'c+d' ]
```

For迴圈

當你要抓取陣列資料時就需要使用到for迴圈，TypeScript有自己的for迴圈寫法但是還是必須知道之前的寫法(TypeScript的寫法只有在TypeScript中可以執行後端或者JavaScript中無法使用)，看到左下的圖片我們需要先設定一個初始值(`let i = 0`)，緊接著一個判斷式讓系統知道這個迴圈是否要繼續執行(`i < 20`)，最後每次執行完要將i遞增(`i++`)這樣當執行完i遞增完為20的時候就不會再去執行迴圈的部份

```
for(let i = 0; i < 20; i++) {  
  console.log(i);  
}
```


For迴圈(TypeScript)

TypeScript自己的迴圈寫法不需要宣告初始值跟條件，但是只能使用在陣列並且沒辦法知道陣列的位置，所以當你需要知道陣列位置的時候就不能使用TypeScript的寫法for 迴圈中的let data為自行命名的變數，而這個data就是陣列的值(左圖 = 右圖)。

```
let arrayData = ['a', 'b', 'c'];  
  
for (let data of arrayData) {  
  console.log(data);  
}
```

```
let arrayData = ['a', 'b', 'c'];  
for (let i = 0; i < arrayData.length; i++) {  
  console.log(arrayData[i]);  
}
```

練習

1. 試著將1~50打印出來。(for迴圈)
2. 承接第一題將偶數去除並且打印出來。
3. 將1~50除以3餘數為2的值打印出來。(for迴圈)
4. 建立右圖的資料並且使用forEach修改小王的age為18。(forEach)
5. 找出陣列裡的最小值([4, 6, 1, 2, 7, 9, 12, 15, 13])
6. 倒著印出字串(你好我是Allen)
7. 將這段文字中的名稱打印出來(我是Allen，你好)

```
// 練習4
let data = [
  {
    name: '小明',
    age: 10
  },
  {
    name: '小陳',
    age: 24
  },
  {
    name: '小王',
    age: 16
  }
]
```

结束

